# DevOps Engineering Assignment

Part A: Theoretical Questions · Part B: Scenario-Based Questions · Part C: Technical / Practical Tasks

## Part A: Theoretical Questions

### 1. What is DevOps Engineering? How does it differ from traditional development and operations roles?

DevOps Engineering is a combination of development (Dev) and operations (Ops) practices that helps teams build, test, deploy, and maintain software efficiently and reliably. It emphasizes collaboration, automation, continuous integration, continuous delivery, monitoring, and continuous improvement.

In traditional organizations, developers write code and hand it over to the operations team for deployment and maintenance. This separation can lead to communication gaps, delayed releases, and difficulties resolving problems.

In DevOps, both teams work together and share responsibility for the software throughout its lifecycle.

| Traditional approach | DevOps approach |
|---|---|
| Development and operations work separately. | Development and operations collaborate. |
| Many tasks are performed manually. | Repetitive tasks are automated. |
| Releases may be infrequent. | Releases can happen frequently and consistently. |
| Problems may lead to blame between teams. | Teams share responsibility for resolving issues. |
| Testing often occurs late. | Testing is integrated throughout the workflow. |

Example: A developer pushes code to GitHub. A CI/CD pipeline automatically tests the code, builds a Docker image, and deploys it to a test environment. Monitoring tools then help the team identify errors after deployment.

### 2. How does DevOps fit into the Software Development Life Cycle (SDLC)?

The Software Development Life Cycle (SDLC) is the process used to plan, develop, test, deploy, and maintain software. DevOps integrates collaboration and automation throughout the SDLC instead of treating deployment and operations as separate activities.

DevOps adds value in the following phases:

1. Planning: Teams collaborate to define requirements, prioritize tasks, and plan releases using tools such as Jira or GitHub Projects.
2. Development: Developers use Git and GitHub for version control, collaboration, and code reviews.
3. Testing: Automated tests run through CI/CD tools such as Jenkins or GitHub Actions to detect defects early.
4. Deployment: Tools such as Docker, Kubernetes, and deployment pipelines help deliver applications consistently.
5. Monitoring and maintenance: Tools such as Prometheus and Grafana help teams monitor performance, detect failures, and improve reliability.

DevOps also encourages feedback from production monitoring to influence future planning and development.

## 3. What values does a DevOps Engineer bring to an organization? Explain any four values with examples.

A DevOps Engineer helps an organization deliver software faster, reduce manual work, improve reliability, and strengthen collaboration.

1. Automation

Automation reduces repetitive manual tasks and the risk of human error.

Example: Using GitHub Actions to automatically run tests whenever a developer pushes new code.

2. Faster delivery

DevOps practices enable teams to release software more frequently through automated build, test, and deployment processes.

Example: A team uses a CI/CD pipeline to deploy a tested application without manually repeating every deployment step.

3. Improved reliability

Automated testing, monitoring, backups, and rollback mechanisms help prevent and recover from failures.

Example: If a new application version causes errors, the team rolls back to the previous stable version.

4. Better collaboration

DevOps encourages developers, testers, security specialists, and operations engineers to share information and responsibility.

Example: During a production incident, team members use a shared incident channel to coordinate troubleshooting and communicate updates.

## 4. Why is prior Software Engineering (SWE) experience helpful before becoming a DevOps Engineer?

Prior Software Engineering experience is helpful because DevOps Engineers frequently work with application code, build systems, testing frameworks, APIs, and deployment configurations.

Software Engineering knowledge helps a DevOps Engineer to:

- Understand application architecture and dependencies.
- Use Git and branching strategies effectively.
- Write scripts in languages such as Python, Bash, or PowerShell.
- Understand debugging, testing, and error handling.
- Create and maintain CI/CD pipelines.
- Communicate effectively with developers.

Example: When a deployment fails because of a missing environment variable, someone with software development knowledge can understand how the application reads configuration and help identify the cause.

However, prior SWE employment is not mandatory for every DevOps role. Beginners can develop the necessary skills through programming practice, projects, and hands-on infrastructure experience.

## 5. Why is networking knowledge important for DevOps roles?

Networking knowledge helps DevOps Engineers deploy applications, configure cloud infrastructure, troubleshoot connectivity problems, and secure communication between services.

Important networking concepts include:

- DNS (Domain Name System): Converts domain names into IP addresses. DNS troubleshooting helps when a website cannot be reached by its domain name.
- Ports: Identify specific services on a machine. For example, port 80 is commonly used for HTTP, port 443 for HTTPS, and port 22 for SSH.
- Protocols: Define how systems communicate. HTTP/HTTPS are used for web traffic, while TCP provides reliable transport.
- IP addressing: Identifies devices and network interfaces. Engineers use IP addresses and subnetting to configure networks, routing, and firewall rules.

Example: If a web application is running but users cannot access it, a DevOps Engineer may check DNS resolution, server IP connectivity, firewall rules, and whether the application is listening on the expected port.

## 6. What soft skills are essential for a DevOps Engineer?

The following soft skills are particularly important:

- Communication: Helps team members share technical information, incident updates, and deployment plans clearly.
- Teamwork: Encourages developers, testers, and operations engineers to solve problems together.
- Problem-solving: Helps engineers analyze logs, identify root causes, and choose appropriate solutions.
- Adaptability: Helps engineers learn new tools and respond to changing requirements.
- Time management: Helps teams prioritize urgent incidents and scheduled work.
- Empathy: Helps engineers support colleagues and avoid blame during failures.

During an incident, clear communication ensures that everyone knows what happened, who is investigating, and what actions are being taken. Teamwork prevents duplicate effort, while problem-solving helps restore service efficiently.

## 7. Explain the importance of patience and a troubleshooting mindset in DevOps with a real-life example.

Patience is important in DevOps because infrastructure and application failures can have several possible causes. Changing configurations without investigating the evidence can make the situation worse.

A troubleshooting mindset involves gathering information, testing hypotheses, identifying the root cause, applying a controlled fix, and verifying the result.

Real-life example:

Suppose a web application becomes unavailable after deployment.

1. The engineer checks application and server logs.
2. The engineer verifies whether the application process is running.
3. The engineer checks the application's listening port and firewall configuration.
4. The logs reveal that the application cannot connect to its database because the database credentials were configured incorrectly.
5. The engineer corrects the configuration securely, restarts or redeploys the application as appropriate, and verifies database connectivity.
6. The team adds a configuration validation check to prevent the same issue from recurring.

Instead of randomly restarting services or changing multiple settings, the engineer uses evidence to find and resolve the actual problem.

# Part B: Scenario-Based Questions

## 1. A new student joins a DevOps course but is unfamiliar with GitHub and cloud platforms. How would you onboard this student efficiently?

I would use a structured onboarding process that introduces essential concepts gradually and provides practical exercises.

1. Assess the student's current knowledge: Identify whether they understand basic command-line operations, programming concepts, Git, and cloud computing.
2. Set up the development environment: Help install Git, a code editor such as VS Code, and the necessary command-line tools.
3. Introduce Git and GitHub: Explain repositories, commits, branches, pull requests, and collaboration through a small practice project.
4. Introduce cloud fundamentals: Explain virtual machines, storage, networking, identity and access management, and cloud billing.
5. Provide documentation: Share beginner-friendly guides, course notes, lab instructions, and troubleshooting resources.
6. Create a practice project: Ask the student to create a simple web application, push it to GitHub, and run it locally.
7. Establish communication channels: Use Slack, Microsoft Teams, or Discord for questions, announcements, and peer support.
8. Provide feedback: Assign small tasks, review progress, and arrange mentoring sessions.

Useful tools: Git, GitHub, VS Code, a cloud provider's learning sandbox, shared documentation, and a team communication platform.

The student should also learn to protect credentials, avoid committing secrets to repositories, and monitor cloud costs before creating resources.

## 2. An application goes down unexpectedly during a production deployment. How would you troubleshoot and handle the situation?

I would prioritize restoring service, communicating clearly, and identifying the cause without introducing additional risk.

Step 1: Acknowledge the incident

Confirm the outage, assess its impact, identify affected users and services, and notify the incident response team.

Step 2: Assess the system

Check application health, server resources, deployment status, logs, metrics, and alerts. Determine whether the failure affects one service or the entire application.

Step 3: Stop the impact and restore service

Pause further deployments. If the new release is responsible and a safe rollback is available, restore the previous stable version. Follow the incident runbook and account for database migration compatibility.

Step 4: Identify the root cause

Compare the new and previous versions, inspect logs, check configuration changes, and verify database, network, and external service connectivity.

Step 5: Verify recovery

Run health checks, test critical user workflows, and confirm that monitoring metrics and error rates have returned to acceptable levels.

Step 6: Document and prevent recurrence

Record the timeline, cause, impact, and resolution. Conduct a blameless post-incident review and introduce preventive measures such as automated tests, staged deployments, and better alerts.

Important principle: Restoring service and reducing user impact come first. Detailed root-cause analysis can continue after the application is stable.

## 3. The team releases features frequently, but system stability is decreasing. How can DevOps practices balance speed and reliability?

The team should improve its delivery process rather than simply reduce the number of releases.

- Continuous Integration (CI): Automatically build and test each code change to catch defects early.
- Automated testing: Use unit, integration, regression, and security tests before releases.
- Continuous Delivery (CD): Standardize deployment steps and use repeatable pipelines.
- Staged deployments: Release changes to a small group of users or a limited percentage of servers before expanding.
- Automated rollback: Restore a known-good version when health checks fail.
- Monitoring and alerting: Track latency, error rates, availability, and resource utilization.
- Infrastructure as Code (IaC): Manage infrastructure configurations consistently with tools such as Terraform.
- Feature flags: Separate deploying code from enabling a feature for all users.
- Blameless reviews: Investigate recurring failures and improve the process instead of blaming individuals.

Example: A team introduces canary deployments, initially directing 5% of traffic to a new version. If error rates increase beyond an agreed threshold, the pipeline stops the rollout and redirects traffic to the stable version.

This approach allows frequent releases while reducing the risk of widespread outages.

### 4. A distributed team struggles with communication during incidents. How can onboarding to communication platforms improve DevOps workflows?

Proper onboarding ensures that all team members know where to communicate, how to report problems, and where to find operational information.

The team should:

1. Establish a shared platform such as Slack or Microsoft Teams.
2. Create dedicated channels for incidents, deployments, announcements, and technical questions.
3. Define severity levels and escalation procedures.
4. Assign clear responsibilities, such as incident commander, technical investigator, and communications lead.
5. Integrate monitoring alerts, CI/CD notifications, and incident management tools into the appropriate channels.
6. Maintain runbooks, contact lists, and incident procedures in a shared documentation system.
7. Train team members on message formatting, status updates, handovers, and escalation expectations.

Example: When a production service fails, the monitoring system posts an alert in the incident channel. The incident commander coordinates the response, engineers share findings, and a designated person posts updates for stakeholders.

This reduces confusion, improves coordination across time zones, and helps the team restore service faster.

# Part C: Technical / Practical Tasks

## Task 1: Local DevOps Demo Application

Objective: Run a simple web application locally and verify that it works.

I would use Python's built-in HTTP server because it is simple to set up and does not require installing a separate web framework.

**Step 1: Check Python installation**

Open a terminal or command prompt and run:

bash
```
python --version
```

If that command is unavailable, try:

bash
```
python3 --version
```

**Step 2: Create a demo application**

Create a folder named `devops-demo`, open a terminal inside it, and create a file named `index.html`.

Add the following HTML:

HTML
Wrap linesCopy codePreview

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>DevOps Demo</title>
</head>
<body>
    <h1>Welcome to My DevOps Demo!</h1>
    <p>My local application is running successfully.</p>
</body>
</html>
```

**Step 3: Start the local web server**

Run the following command from inside the `devops-demo` folder:

bash
```
python -m http.server 8000
```

If necessary, use `python3` instead of `python`.

You should see output indicating that the HTTP server is serving files on port 8000.

**Step 4: Verify the application**

Open your browser and visit:

Local application URL

**http://localhost:8000**

Accessible from your own computer while the server is running.

Copy

The browser should display the heading “Welcome to My DevOps Demo!” and the message confirming that the application is running.

You can also verify it from another terminal:

bash
```
curl http://localhost:8000
```

A successful response should contain the HTML from your `index.html` file.

Illustrative reference only. Capture a screenshot of your own browser and terminal after running the commands.

**Step 5: Explain how local applications help DevOps workflows**

Running applications locally helps DevOps Engineers to:

- Test code and configuration changes before deployment.
- Identify errors early, reducing the risk of production failures.
- Practice containerization using Docker.
- Develop and test CI/CD pipelines.
- Reproduce bugs in a controlled environment.
- Verify application behavior before deploying to a cloud platform.

Result: A basic web application runs locally on port 8000 and can be verified through a browser or `curl`.

## Task 2: Exposing the Local Application Using ngrok

Objective: Make the local application temporarily accessible through a public internet URL.

ngrok creates a secure tunnel from a public endpoint to a service running on your computer.

Step 1: Install ngrok

Visit the official website: ngrok



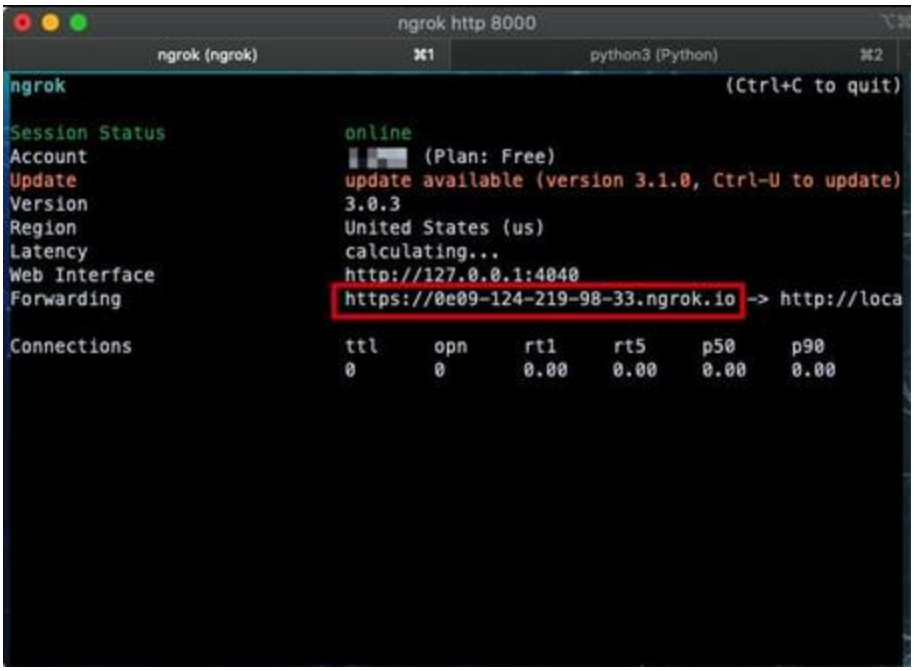

Create an account if required, install ngrok for your operating system, and follow the dashboard instructions to configure your authentication token.

Step 2: Keep the local server running

In your first terminal, make sure the Python server is still running:

bash

```
python -m http.server 8000
```




Step 3: Start the ngrok tunnel

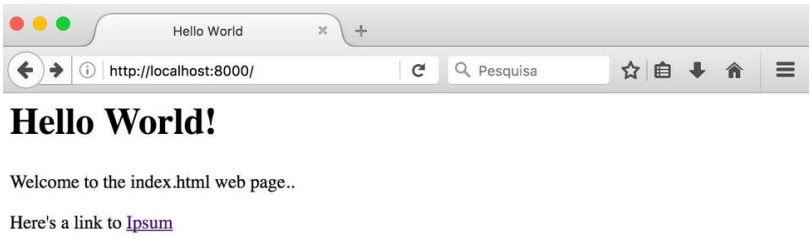
Open a second terminal and run:

bash
```
ngrok http 8000
```




ngrok should display a forwarding address similar to `https://example-name.ngrok-free.app`. The actual address will be generated by ngrok and may be different.

Step 4: Verify the public URL

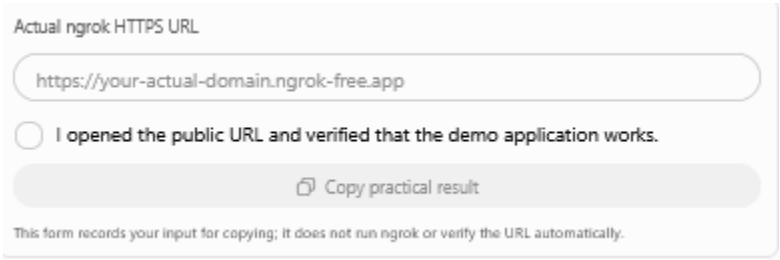



Copy the actual HTTPS forwarding URL shown in your terminal and open it in a browser, preferably from another device or network.

Confirm that the same demo page appears. Keep both the Python server and ngrok running while testing.

**Public URL capture**

After you run ngrok, record your real URL below.




Actual ngrok HTTPS URL

☐ I opened the public URL and verified that the demo application works.

Copy practical result

This form records your input for copying; it does not run ngrok or verify the URL automatically.

Screenshot evidence to include in your submission:

- Screenshot 1: The local application open at `http://localhost:8000`.
- Screenshot 2: The terminal showing the running Python HTTP server.
- Screenshot 3: The ngrok terminal displaying the actual HTTPS forwarding URL.
- Screenshot 4: The demo application successfully opened using that public URL.

**One real-world use case for exposing a local application**

A developer can share a locally running application with a remote teammate or client for testing and feedback before deploying it to production.

For example, a developer working on a payment integration could use a temporary public tunnel to receive test webhooks from an external service while the application runs on their laptop.

Security note: A public tunnel may expose your local application to anyone who has access to its URL. Do not expose confidential files, administrative interfaces, production credentials, or sensitive services. Use test data and stop the tunnel when finished.

## Conclusion

DevOps Engineering combines software development, infrastructure management, automation, testing, deployment, monitoring, and collaboration. Its main goal is to deliver software quickly without sacrificing quality, security, or reliability.

The practical tasks demonstrate two foundational DevOps concepts: testing an application in a local environment and using a secure tunnel to make that application accessible externally. The commands and steps above provide a reproducible demonstration; the actual application execution, ngrok URL, and screenshots still need to be produced on your computer.